Locality-First Messaging: Decoupling Availability and Consensus

Narayan Krishna R
nrajeshk@usc.edu

Kriti Bhattacharya
kritibha@usc.edu

VenkataramanVenkatasubaniam
vv70613@usc.edu

Abstract

We present a distributed log replication system that combines gossip-based synchronization with deterministic ordering and control-plane authorization. Nodes exchange summaries to efficiently converge without redundant data transfer, while Lamport clocks ensure consistent ordering across replicas. An epoch-based mechanism enforces membership by rejecting invalid updates before merge. Our evaluation shows fast convergence (~100 ms), no duplication under replay, and correct enforcement of access constraints.

1. Introduction

Distributed systems are widely used to maintain replicated state across multiple nodes, whether for logging, storage, or coordination. A fundamental challenge in such systems is ensuring that all nodes eventually converge to a consistent view of the data, even in the presence of concurrent updates, delayed communication, and node failures. In practice, this problem becomes more complex when nodes operate independently and communicate asynchronously, as inconsistencies can arise from out-of-order message delivery or repeated transmissions.

Consider a simple scenario where multiple nodes maintain a shared log of messages. If one node appends a new entry while another node is temporarily offline, the system must later reconcile these states without duplicating entries or introducing inconsistencies. Additionally, if nodes repeatedly exchange updates, the system must avoid reapplying the same changes multiple times. Without careful design, such behavior can lead to divergent logs or inefficient communication overhead.

In this project, we design and implement a distributed log replication system that addresses these challenges through a combination of gossip-based synchronization, deterministic ordering, and control-plane enforced authorization. Nodes communicate using a lightweight gossip protocol, exchanging summaries of their state to determine whether synchronization is necessary. This approach, often referred to as anti-entropy, reduces unnecessary data transfer while ensuring eventual convergence.

To guarantee consistent ordering of updates across nodes, we employ Lamport clocks, which assign logical timestamps to each operation. These timestamps, combined with deterministic tie-breaking, ensure that all nodes process updates in the same order, even under concurrent conditions. Furthermore, we integrate an epoch-based authorization mechanism, where each message is validated against a control-plane service before being accepted. This prevents stale or unauthorized updates from being incorporated into the system.

Our implementation demonstrates that these techniques can be combined into a simple yet effective system that achieves eventual consistency, avoids duplication, and enforces correctness constraints. Through a series of experiments, we evaluate the system's convergence behavior, resistance to repeated updates, and ability to reject invalid inputs, providing evidence of its correctness and robustness.

2. System Design

2.1 Overview

The system is designed around a key architectural principle: decoupling availability from consistency. Instead of enforcing global ordering at the time of message ingestion, nodes are allowed to accept local writes immediately and reconcile state asynchronously. This design enables the system to remain responsive under network delays, partitions, and intermittent connectivity, while still ensuring eventual consistency.

At a high level, the system consists of a set of peer nodes that maintain replicated append-only logs, a lightweight client that initiates updates, and a control-plane service responsible for enforcing membership rules. This approach follows a locality-first model, where operations are applied locally and propagated later, rather than relying on a centralized coordinator or strict global ordering. As motivated in our proposal, this design is particularly useful in scenarios such as offline messaging or partitioned networks, where immediate availability is more critical than strict consistency .

2.2 Data Plane: Gossip-Based Synchronization

The data plane is responsible for maintaining and synchronizing replicated logs across nodes. Each node stores messages in an append-only log and exchanges state with peers using a gossip protocol.

Instead of transmitting full logs, nodes first exchange compact summaries, which include metadata such as the number of entries and the maximum logical timestamp. These summaries allow nodes to determine whether their states differ. If the summaries match, no further action is taken; otherwise, the node sends the missing log entries.

This mechanism implements an anti-entropy synchronization strategy, which ensures nodes eventually converge while minimising redundant communication.

A key property of this approach is that it is inherently tolerant to unreliable networks. Messages may be delayed, duplicated, or reordered, but repeated synchronization attempts eventually bring all nodes to a consistent state.

2.3 Deterministic Merge and Ordering

To ensure that all nodes converge to the same log, the system employs a deterministic merge strategy. When nodes receive updates from peers, they perform a set-union merge of entries, eliminating duplicates and incorporating new data.

However, simple set union is not sufficient to guarantee consistent ordering across nodes. To address this, each entry is assigned a Lamport timestamp, which captures the causal ordering of events. When merging logs, entries are sorted by:

1. Lamport timestamp, and
2. a deterministic tie-breaker (e.g., message identifier).

This ensures that even when multiple nodes generate updates concurrently, all replicas resolve conflicts in the same way, leading to identical log orderings.

2.4 Control Plane: Membership and Authorization

While the data plane prioritizes availability, the control plane is responsible for enforcing membership and access control policies. This separation allows the system to maintain strong guarantees for authorization without compromising the responsiveness of local operations.

The control plane maintains an Access Control List (ACL) and exposes an authorization interface. Each message includes an epoch value, representing the version of the membership state under which it was generated. When a node receives a message, it queries the control-plane service to verify whether the message's epoch is still valid.

If the epoch is valid, the message is accepted and merged into the log. Otherwise, it is rejected and discarded. This ensures that evoked users cannot inject updates and stale or delayed messages are filtered.

By separating policy definition (control plane) from enforcement (data plane), the system achieves both flexibility and correctness.

2.5 Hybrid AP/CP Model

The system follows a hybrid model that combines availability-first (AP) behavior for data operations with consistency-oriented (CP) behavior for membership control.

- The data plane operates in an AP mode:
    - local writes always succeed,
    - nodes continue operating under partitions,
    - synchronization happens asynchronously.
- The control plane operates in a CP mode:
- membership changes require coordination,
- authorization decisions are globally consistent.

This hybrid design enables the system to provide strong guarantees where necessary (membership correctness) while maintaining high availability for regular operations. As outlined in the proposal, this separation allows the system to tolerate network failures and still ensure that all nodes eventually converge without violating access constraints .

2.6 Fault Tolerance and Recovery

The system is designed to tolerate crash-stop failures and intermittent connectivity. Nodes can fail and later rejoin the system without requiring global coordination.

When a node recovers, it synchronizes with peers using the same gossip-based mechanism. Since logs are append-only and merges are idempotent, the node can safely replay missed updates without duplicating entries. This replay-based recovery ensures that nodes eventually catch up to the latest state.

Additionally, because synchronization is driven by summaries rather than strict coordination, nodes can recover from partitions and resume normal operation without requiring a centralized leader for data replication.

3. Implementation

3.1 System Components

The system is implemented as a set of modular components corresponding to the data and control planes:

- Log Store (`log_store.cpp`): Maintains the append-only log and implements merge, append, and hashing operations.
- Lamport Clock (`lamport_clock.cpp`): Generates logical timestamps for deterministic ordering.
- Data Plane Service (`data_plane_service.cpp`): Implements the `SyncLogs` RPC, handling synchronization, validation, and merging.

- Client (`log_client.cpp`): Generates updates and initiates gossip-based synchronization.
- Authorization Service (`fake_auth_server.cpp`): Validates message epochs and enforces access control.
- Server (`log_server.cpp`): Hosts the gRPC service and binds the data plane to network endpoints.

These components are loosely coupled, allowing independent evolution of data-plane and control-plane logic.

3.2 Communication via gRPC

Nodes communicate using gRPC through a single RPC:

SyncLogs(SyncLogsRequest) → SyncLogsResponse

Requests include the sender identity, a log summary, and optionally log entries. Responses include a success flag, the receiver's Lamport time, and its summary. This unified interface supports both lightweight summary exchange and full synchronization.

3.3 Anti-Entropy Synchronization

Synchronization follows a two-phase process:

1. **Summary Exchange**
   Nodes exchange summaries to detect state differences.
2. **Conditional Log Transfer**
   If summaries differ, missing entries are sent; otherwise, no transfer occurs.

This design reduces redundant communication while ensuring eventual convergence, even under delays or repeated synchronization.

3.4 Merge and Idempotence

The merge operation is **idempotent and deterministic**. Incoming entries are added only if not already present, using message identifiers for deduplication. Entries are then ordered by Lamport timestamp with deterministic tie-breaking.

This ensures:

- no duplication under repeated synchronization, and
- identical ordering across nodes after convergence.

3.5 Epoch-Based Authorization

Authorization is enforced during synchronization via the `checkAuth` function. Each message is validated by querying the control-plane service with the sender ID and epoch.

Invalid messages are discarded before merging, ensuring that:

- revoked users cannot inject updates,
- stale messages are filtered, and
- membership policies are consistently enforced.

3.6 Fault Tolerance and Recovery

The system supports recovery from crash-stop failures through replay-based synchronization. Restarted nodes exchange summaries with peers and retrieve missing entries.

Because merges are idempotent, updates can be safely reapplied without duplication. The append-only log further simplifies recovery by avoiding complex state reconstruction.

3.7 Testing and Validation

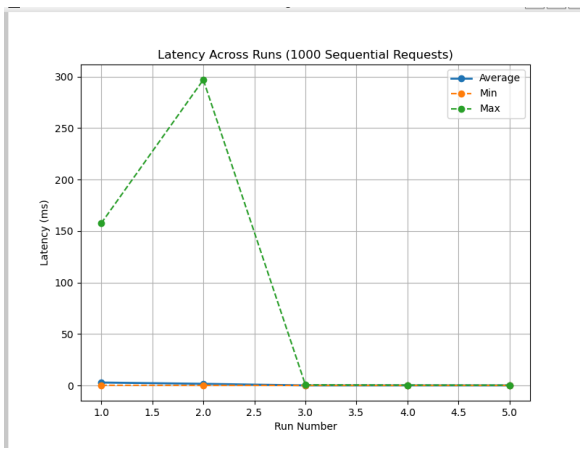
The implementation is validated through unit and integration tests:

- Log store tests for merge, ordering, and deduplication
- Lamport clock tests for timestamp correctness
- gRPC tests for end-to-end communication
- Raft and crash tests for control-plane behavior

Manual experiments further confirm convergence, replay safety, and correct rejection of invalid messages.

```
Run 2 | avg=1.62278 ms | min=0.069236 ms | max=296.965 ms
Run 3 | avg=0.10195 ms | min=0.069407 ms | max=0.59768 ms
Run 4 | avg=0.0989222 ms | min=0.069956 ms | max=0.39922 ms
Run 5 | avg=0.0905869 ms | min=0.068615 ms | max=0.282861 ms

=== FINAL ===
Avg: 0.956129 | Min: 0.068615 | Max: 296.965
```



4. Performance Evaluation

We evaluate our system along the key dimensions outlined in our proposal: convergence, ordering consistency, membership enforcement, and performance. Our experiments are conducted on a local multi-process setup, where each node runs as an independent process and communicates via gRPC.

4.1 Convergence

To evaluate convergence, we deploy three nodes and repeatedly synchronize logs using the gossip-based protocol. Starting from an empty state, one node generates updates while others receive them through the synchronization process.

We observe that after a small number of synchronization rounds, all nodes reach identical states, as verified by matching log sizes and hash values. Furthermore, convergence is maintained even when synchronization is repeated multiple times, demonstrating that the system is robust to redundant communication.

This confirms that the anti-entropy mechanism ensures eventual consistency, even under asynchronous communication and repeated message exchange.

4.2 Replay and Idempotence

We evaluate the system's behavior under repeated synchronization by invoking the client multiple times with the same set of log entries. The first synchronization transfers the required entries to peers, while subsequent runs perform only summary exchanges.

We observe that:

- no duplicate entries are introduced, and
- subsequent synchronization attempts correctly identify that nodes are already up-to-date.

This demonstrates that the merge operation is idempotent, and that the anti-entropy protocol avoids unnecessary data transfer once convergence is achieved.

4.3 Membership Enforcement

To evaluate membership enforcement, we simulate both valid and invalid message scenarios using epoch-based authorization.

In the valid case, all messages are accepted and merged successfully. In the invalid case, messages with outdated or incorrect epoch values are

rejected by the data plane after verification with the control-plane service.

We observe that:

- invalid messages are consistently dropped before entering the log, and
- valid messages continue to propagate normally across nodes.

Importantly, rejecting invalid messages does not prevent the system from converging on the remaining valid state. This demonstrates that the integration of control-plane authorization does not interfere with data-plane consistency.

4.4 Ordering Consistency

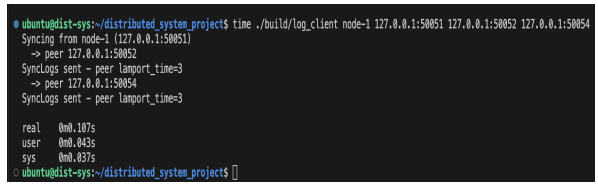
We evaluate ordering consistency by introducing concurrent updates across nodes and verifying that all replicas resolve these updates in the same order.

Using Lamport clocks combined with deterministic tie-breaking, all nodes produce identical log orderings after synchronization. This confirms that the system achieves deterministic total ordering, even under concurrent updates.

4.5 Convergence Latency

We measure convergence latency as the time required for nodes to synchronize their logs after updates are introduced. We approximate this by measuring the execution time of the client-triggered synchronization process across multiple nodes.

In our experiments, a single synchronization round across three nodes completed in approximately **100–110 ms**, as measured using system timing utilities. A representative run produced:



```
ubuntu@dist-sys:~/distributed_system_project$ time ./build/log_client node-1 127.0.0.1:50051 127.0.0.1:50052 127.0.0.1:50054
  Syncing from node-1 (127.0.0.1:50051)
    -> peer 127.0.0.1:50052
  SyncLogs sent - peer lamport_time=3
    -> peer 127.0.0.1:50054
  SyncLogs sent - peer lamport_time=3

  real    0m0.107s
  user    0m0.043s
  sys     0m0.037s
ubuntu@dist-sys:~/distributed_system_project$
```

These results indicate that convergence is achieved quickly under local execution conditions. The use of summary-based synchronization reduces unnecessary data transfer, allowing nodes to determine whether updates are needed before sending full log entries. As a result, subsequent synchronization attempts complete significantly faster, as only lightweight summary exchanges are performed.

While these measurements are taken in a single-machine environment and do not reflect network variability, they demonstrate that the system converges efficiently and scales with repeated synchronization attempts.

4.6 Fault Tolerance and Recovery

We evaluate fault tolerance by simulating node restarts and rejoining behavior. After a node is restarted, it reconnects to peers and performs synchronization using the same gossip mechanism.

We observe that:

- the restarted node successfully retrieves missing entries, and
- the system converges without requiring global coordination.

This demonstrates that the system supports recovery through replay and gossip-based synchronization, consistent with the design goals.

5. Discussion and Limitations

While the system achieves its primary goals of availability, convergence, and controlled

membership enforcement, there are several limitations and trade-offs worth noting.

First, our implementation operates in a single-machine, multi-process environment, where all nodes communicate over localhost. While this setup is sufficient to validate correctness and demonstrate system behavior, it does not capture real-world network conditions such as latency variability, packet loss, or large-scale deployment challenges. As a result, performance measurements, particularly latency, should be interpreted as indicative rather than definitive.

Second, the system relies on a simplified authorization service to enforce membership rules. The current implementation uses a mock or "fake" authentication server, which deterministically accepts or rejects messages based on epoch values. It does not reflect the complexity of a real-world security system, which would involve cryptographic verification, secure key distribution, and resistance to adversarial behavior.

Third, the system does not fully explore Byzantine fault tolerance or adversarial conditions. While invalid messages are rejected through epoch validation, the system assumes that nodes themselves behave correctly and does not include mechanisms such as message signing or quorum-based validation. Incorporating such features would strengthen the system's robustness but would also increase complexity and overhead.

6. Future Work

Several directions can be explored to extend this work. A natural next step would be to deploy the system in a distributed environment across multiple machines to evaluate performance under realistic network conditions. Additionally, integrating stronger security mechanisms, such as digital signatures and authenticated communication channels, would improve robustness against adversarial behavior.

Another avenue for improvement is enhancing the control-plane integration, potentially using a fully implemented consensus protocol to manage membership changes dynamically. Finally, expanding the system to support richer data models or more general CRDTs could broaden its applicability beyond append-only logs.

7. Code

You can find our C++ Implementation on:
https://github.com/CSCI546-26sp/final-project-doohickey-engineering